Maestro - AI-first higher education

Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

FE101

Why TypeScript?

Today

Welcome to your next lesson: Why TypeScript?

I'm setting up your lesson with what you've just learned, what's coming up, and how you learn best.

Step by step, you’re moving forward.

Ron, think back to your last lesson with fetch() and async code 🤓

If you make a tiny mistake in JavaScript like this:

Typescript
```
const response = await fetch("htts://api.example.com/data"); // typo in "https"
const data = await response.json();
console.log(data);
```
JavaScript won’t complain until you run it — then at runtime you get an error. Until that moment, everything looks fine.

TypeScript is about catching a different kind of mistake before you run the code — like a spell-checker for your program.

To start simple: in your own words, how much have you seen or used TypeScript before?

A) Never used it
B) Played with it a tiny bit
C) Used it in a real project
a

Nice, starting from zero is perfect for this lesson 🚀

Let me connect one idea first, step by step.

In JavaScript or Python you can do this:

Typescript
```
let x = 5;
x = "hello";
```
The language is fine with changing x from a number to a string at runtime. It only blows up later if some code expects x to still be a number.

In strongly typed languages (like TypeScript when types are turned on), you can say:

“x must always be a number.”

Then the editor/compiler will complain before you run the program if you try to assign a string.

Does that basic idea make sense: some languages are loose about types at runtime, and some are stricter and check types earlier?

Yes, that makes sense

Great ☑

Let’s make this concrete with TypeScript.

In TypeScript you can lock in what kind of value a variable should hold, like this:

Typescript
```
let age: number = 30;
```
Read it as:

"age is a variable, and it must always be a number."

If later you write:

Typescript
```
age = "thirty";
```
TypeScript will flag this before running the code, because "thirty" is a string, not a number.

Your turn (no code editor yet, just thinking):

If I write in TypeScript:

Typescript
```
let username: string = "ron";
```
Is TypeScript expecting username to always hold:

a number, or
a string?

Just answer: “number” or “string”.

string

Exactly — string 🎯

So when you write:

Typescript
let username: string = "ron";
you're telling TypeScript:

"username should always be a text value."

Next step, I want you to actually feel this in the code editor.

Open the code editor for this lesson.
Type this code:
Typescript
let score: number = 100;
console.log(score);
Hit Run (you don't need to submit yet).

Do that now, and tell me: did it run without errors?

I ran it successfully

Nice ☑ that's your first bit of real TypeScript.

Let's push it one small step.

Right now you have:

Typescript
let score: number = 100;
console.log(score);
This is valid because:

score is annotated as a number
100 is a number
Now, still in the editor, change only the second line to this:

Typescript
score = "high score!";
console.log(score);
So the full code becomes:

Typescript
let score: number = 100;
score = "high score!";
console.log(score);
Hit Run (and if you like, then Submit code so I can see it).
Don't worry if it still prints something — focus on the idea.

After you run it, tell me:

Do you think this assignment matches the type number, or does it break the rule we set for score?

Typescript
let score: number = 100;
score = "high score!";
console.log(score);
Perfect, Ron — this is exactly what I wanted you to hit ✡

TypeScript says:

Type 'string' is not assignable to type 'number'.

Step by step, what happened:

You told TS: score must be a number.
Then you tried to assign "high score!" which is a string.
Static TypeScript check catches the mismatch before running the program.

This is what people mean by:

TypeScript is strongly / statically typed and does static analysis to find bugs before runtime.

Quick check for you, in your own words (one short sentence):

Why did TypeScript complain about score = "high score!";?

score was explicilty defined as a variable that can only hold numbers but an attempt was made to assign a string to it

Exactly right 💯

You declared score as number-only, then tried to give it a string, so TypeScript stopped you.

That’s strong/static typing in action.

Let’s add one more type so you see this pattern clearly: boolean (true/false).

In the editor, write and Run this code (then Submit):

Typescript
```
let isLoggedIn: boolean = true;
console.log(isLoggedIn);
```
Don’t change anything yet—just this.

After you run it, tell me: did it run fine?

it ran fine

Nice 👌

You just successfully used boolean with isLoggedIn.

Quick recap with a simple visual:

Variable	Type	Example value
username	string	"ron"
score	number	100
isLoggedIn	boolean	true / false

TypeScript’s rule is:

once you say the type, assignments have to match it, or you get an error before running.

Next tiny step: I want you to trigger the boolean error on purpose.

In the same file, change your code to:

Typescript
let isLoggedIn: boolean = true;
isLoggedIn = "yes";
console.log(isLoggedIn);
Run it.
Then Submit the code so I can see the output/error.

Don’t explain yet—just run and submit, then we’ll talk through what TypeScript is doing.

Typescript
let isLoggedIn: boolean = true;
isLoggedIn = "yes";
console.log(isLoggedIn);
Nice work triggering that error on purpose 💣

TypeScript’s message:

Type 'string' is not assignable to type 'boolean'.

Same pattern as before:

You told TS: isLoggedIn must be a boolean (true/false)
You tried to assign "yes" which is a string
Static analysis caught it before runtime

So far you’ve:

Used string, number, and boolean annotations
Seen TypeScript catch invalid assignments early
Now I want to show you a quick visual overview of why TypeScript is useful — a short 100□second video.

Want to watch that now? (It will just pop up as a video player.)

Yes, show me the video

Take your time with the video 📽

It gives a fast, high□level picture of what you’ve already started doing in code.

When you’re done watching, tell me one thing you picked up from it — even just a short phrase like “catches bugs earlier” or “TypeScript is JS with types”.

i like that it adds the ability for VS Code to add auto complete hints as your typing code, giving himts a to the proper data types allowed

Love that you noticed that 👀

Yeah, TypeScript’s types power up VS Code:

better autocomplete
smarter hints about what values are allowed
warnings when you’re about to use the wrong type

That’s another benefit of strong typing: not just fewer bugs, but better tooling while you type.

Let’s lock in one key idea now, very simply:

Static analysis = the TypeScript checker looking at your code before it runs to find type problems (like assigning a string where a number/boolean is expected).

So, one quick check:

In your own words, is static analysis happening while the program runs or before you run it?

Before it runs

Exactly — before it runs ☑

That’s the core of how TypeScript catches bugs early.

Now I want to check one bigger idea: strong/static typing vs JavaScript/Python.

Here’s the picture:

JavaScript / Python

Let variables change type at runtime
Errors often show up only when you run the code
TypeScript (with types)

You declare the allowed type (number, string, boolean)
The checker yells before run if you break that rule

🎯 One-sentence challenge for you:

Can you describe the main advantage of this strong/static typing in TypeScript, compared to plain JavaScript/Python?

(Keep it short, like: “It helps by …”)

it helps by catching erorors during the coding process and not waiting until the prodction code blows up on the end user, potentially costing the company money

That’s a strong, real-world answer 💼🔥

Yes: fewer surprises in production, happier users, less money burned on dumb bugs.

You’ve basically hit all three goals for this lesson:

You explained the benefit of strong/static typing vs JS/Python (catching mistakes early, protecting production).
You saw how TypeScript uses static analysis to flag wrong assignments before runtime.
You wrote variables with explicit types and triggered TS errors on purpose (number vs string, boolean vs string).
Before we wrap this lesson:

Do you feel ready to move on to the next lesson, or do you want one more quick practice with string / number / boolean annotations?

I’m ready to move on

Awesome work today, Ron 🎯

Quick recap of what you nailed:

Strong/static typing: TypeScript lets you say “this variable must be a number / string / boolean”, unlike JavaScript/Python where types can change at runtime.
Static analysis: The TS checker (and VS Code) scans your code before it runs, warning you when you assign the wrong type.
Explicit annotations: You wrote code like let score: number = 100; and saw TypeScript errors when you tried to assign "high score!" or "yes" to the wrong types.

You also connected this to the real world: preventing production crashes and saving money. That’s exactly how a pro thinks 👌

You’re all set to continue.

When you’re ready, hit Next lesson in the Learn tab, and we’ll keep building on this TypeScript foundation.

```
let isLoggedIn: boolean = true;
isLoggedIn = "yes";
console.log(isLoggedIn);
```